

ArchPortal

Architecture Firm Management Portal

Full Planning & Architecture Document

Prepared for Claude Code Implementation

Version 1.0 · Django + PostgreSQL · Tailwind CSS

Table of Contents

#	Section	Description
1	Domain Research	How architecture firms operate
2	System Overview	Goals, users, constraints
3	Tech Stack	Full stack decision & rationale
4	System Architecture	ASCII diagram + layer breakdown
5	Database Schema	All models & relationships
6	Module Specifications	4 core modules detailed
7	API Endpoints	All routes & payloads
8	Project File Structure	Complete folder tree

9	Implementation Roadmap	Phases & task checklist
10	Claude Code Instructions	How to implement step by step

1. Domain Research — How Architecture Firms Operate

Before writing a single line of code, understanding the business domain is critical. Architecture firms operate very differently from generic service companies. Their workflows are project-centric, milestone-driven, and people-intensive. Below are key findings.

1.1 Project Lifecycle Phases

Phase	Description	Typical Duration
Schematic Design (SD)	Initial concept sketches, site analysis, client brief review	2–4 weeks
Design Development (DD)	Detailed drawings, structural decisions, material selection	4–8 weeks
Construction Documents (CD)	Full working drawings, specs for contractor bids	6–12 weeks
Bidding / Tender	Soliciting contractor bids, reviewing proposals	2–4 weeks
Construction Administration (CA)	Site visits, RFIs, change orders, progress monitoring	Duration of build
Project Closeout	Punch list, as-built drawings, handover documentation	2–4 weeks

1.2 Roles Inside a Firm

- **Principal / Managing Director** — Owns the firm, signs contracts, approves budgets, final accountability.
- **Project Architect** — Leads individual projects end-to-end. Licensed professional (B.Arch / M.Arch).
- **Junior Architect / Draftsperson** — Produces drawings under supervision. May hold Part-1 or Part-2 qualification.
- **Site Supervisor** — On-site representative during CA phase. Checks quality and compliance.
- **Accountant / Finance Manager** — Handles invoicing, expense tracking, payroll, client billing.
- **Office Manager / Admin** — Manages scheduling, document control, vendor relationships.

1.3 Billing Patterns

Architecture firms commonly use **milestone-based billing** — invoices are raised at the completion of each project phase (e.g., 20% on SD completion, 30% on DD, 30% on CD, 20% on CA). Some firms bill hourly. Overdue invoices are a real pain-point — the portal must surface them prominently.

1.4 Pain Points the Portal Must Solve

- No centralised view of which architect is working on what project (resource conflicts).
- Missed deadlines because project phases are tracked in spreadsheets.
- Invoices falling through the cracks — no system to flag overdue payments.
- Management cannot see project health at a glance without asking each architect.
- No audit trail for project status changes or assignment changes.

2. System Overview

2.1 Project Goal

ArchPortal is an internal web-based management system for small-to-medium architecture firms. It centralises project tracking, architect allocation, and billing operations. The system reduces manual coordination overhead and gives management real-time operational visibility.

2.2 User Roles & Permissions

Role	Access Level	Key Permissions
Admin	Full Access	All CRUD operations, user management, all reports, delete records
Architect	Limited Access	View own projects, update phase status, view own invoices
Accountant	Billing Access	Full billing module, view all projects (read-only), create/edit invoices

2.3 Non-Functional Requirements

- Responsive UI — usable on laptop and tablet.
- Role-based access control enforced server-side (not just front-end hiding).
- All forms validated both client-side (HTML5) and server-side (Django forms).
- Passwords hashed with PBKDF2 (Django default).
- All database queries use Django ORM — no raw SQL for security.
- Static files served via WhiteNoise in development/staging.
- Environment variables for all secrets (SECRET_KEY, DB credentials).

3. Tech Stack — Full Decision & Rationale

Layer	Technology	Version	Rationale
Backend Framework	Django	5.x	Batteries-included: auth, ORM, admin, forms. Fastest path to working C
Database	PostgreSQL	16.x	Production-grade, ACID compliant. Better than SQLite for multi-user.
DB Adapter	psycopg2-binary	2.9.x	Standard Django-PostgreSQL connector.
Frontend CSS	Tailwind CSS	3.x (CDN)	Utility-first. No build step needed for prototypes.
UI Components	Flowbite	2.x (CDN)	Pre-built Tailwind components: tables, modals, dropdowns.
Charts	Chart.js	4.x (CDN)	Lightweight. Works with Django template tags.
Icons	Heroicons	CDN SVG	Consistent icon set matching Tailwind aesthetic.
Environment	python-decouple	3.8	Manages .env file for secrets.
Static Files	WhiteNoise	6.x	Serves static files without Nginx in dev.
Forms	django-crispy-forms	2.x	Renders Django forms with Tailwind classes cleanly.
Auth	Django built-in	—	django.contrib.auth handles login/logout/passwords.
Dev Server	Django runserver	—	Adequate for development and evaluation.
Python	Python	3.11+	Required by Django 5.x.

3.1 Why Django over Next.js + FastAPI

Consideration	Django Monolith wins because...
Time to first feature	Django ships auth, admin, ORM out of the box. Next.js+FastAPI needs wiring.
Auth complexity	django.contrib.auth handles sessions, CSRF, password hashing with zero config.
Database migrations	django makemigrations / migrate is the gold standard.
CRUD forms	Django ModelForms auto-generate validated HTML forms from models.
Admin panel free	Django Admin gives a full back-office UI for free — great for demos.
Deployment simplicity	Single Python process. No two servers, no CORS, no JWT tokens.

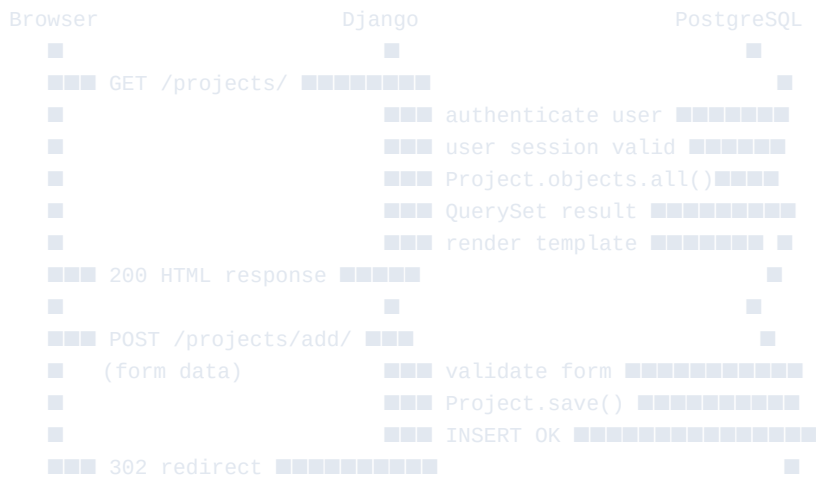
4. System Architecture

4.1 High-Level ASCII Architecture Diagram





4.2 Request-Response Flow

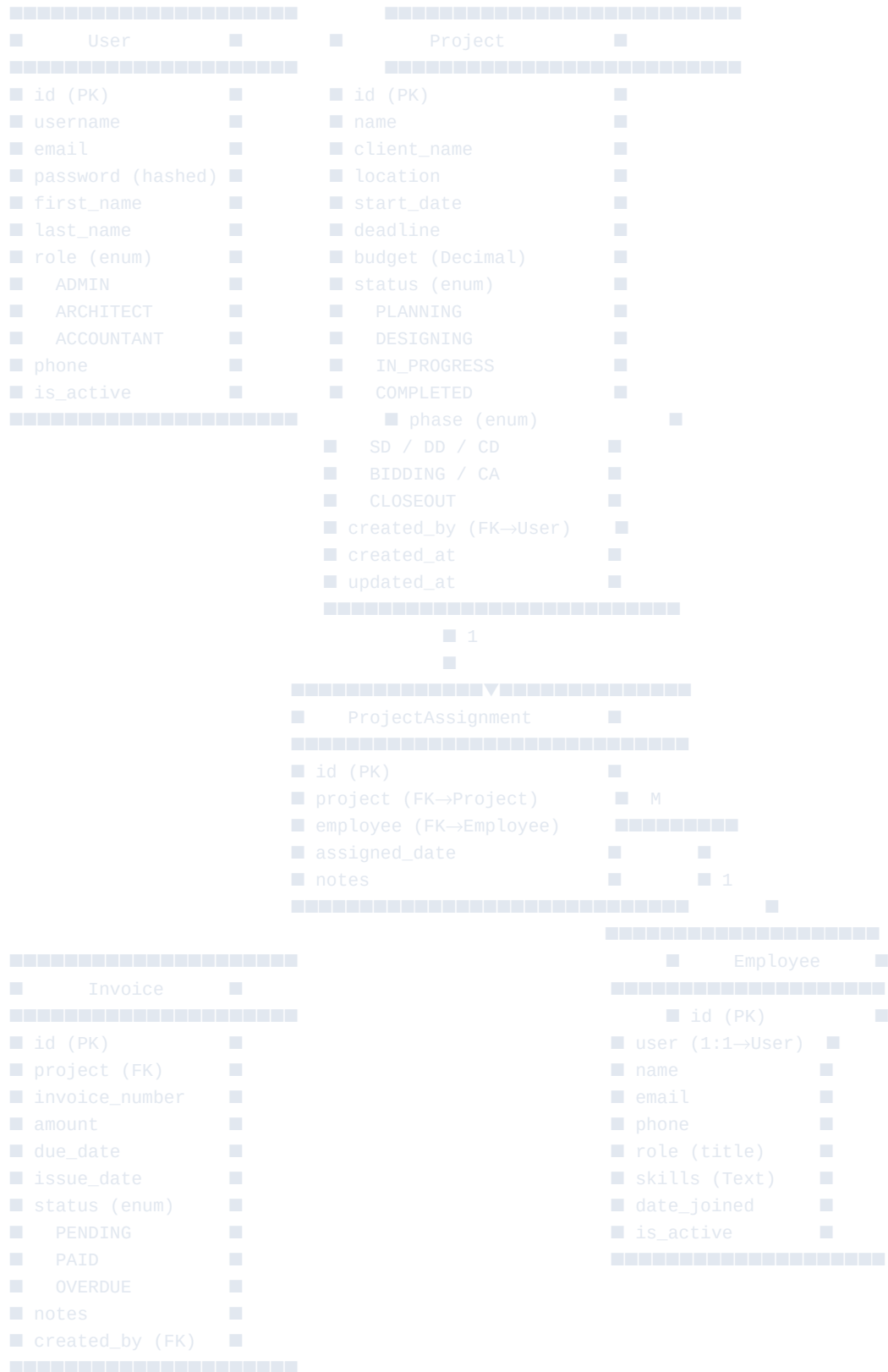


4.3 Application Module Map



5. Database Schema — All Models & Relationships

5.1 Entity Relationship Diagram (ASCII)



5.2 Model Definitions for Claude Code

Paste these directly into each app's models.py:

accounts/models.py

```
from django.contrib.auth.models import AbstractUser
from django.db import models

class User(AbstractUser):
    class Role(models.TextChoices):
        ADMIN = 'ADMIN', 'Admin'
        ARCHITECT = 'ARCHITECT', 'Architect'
        ACCOUNTANT = 'ACCOUNTANT', 'Accountant'

    role = models.CharField(max_length=20, choices=Role.choices,
                           default=Role.ARCHITECT)
    phone = models.CharField(max_length=20, blank=True)

    def is_admin(self): return self.role == self.Role.ADMIN
    def is_architect(self): return self.role == self.Role.ARCHITECT
    def is_accountant(self): return self.role == self.Role.ACCOUNTANT
```

projects/models.py

```
from django.db import models
from django.conf import settings

class Project(models.Model):
    class Status(models.TextChoices):
        PLANNING = 'PLANNING', 'Planning'
        DESIGNING = 'DESIGNING', 'Designing'
        IN_PROGRESS = 'IN_PROGRESS', 'In Progress'
        COMPLETED = 'COMPLETED', 'Completed'

    class Phase(models.TextChoices):
        SD = 'SD', 'Schematic Design'
        DD = 'DD', 'Design Development'
        CD = 'CD', 'Construction Documents'
        BIDDING = 'BIDDING', 'Bidding/Tender'
        CA = 'CA', 'Construction Administration'
        CLOSEOUT = 'CLOSEOUT', 'Project Closeout'

    name = models.CharField(max_length=200)
    client_name = models.CharField(max_length=200)
    location = models.CharField(max_length=300)
    start_date = models.DateField()
    deadline = models.DateField()
    budget = models.DecimalField(max_digits=14, decimal_places=2)
    status = models.CharField(max_length=20, choices=Status.choices,
                             default=Status.PLANNING)
    phase = models.CharField(max_length=20, choices=Phase.choices,
                             default=Phase.SD)
    description = models.TextField(blank=True)
    created_by = models.ForeignKey(settings.AUTH_USER_MODEL,
                                   on_delete=models.SET_NULL, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
```

```

        ordering = ['-created_at']

    def __str__(self):
        return f"{self.name} - {self.client_name}"

    def is_overdue(self):
        from django.utils import timezone
        return self.deadline < timezone.now().date() \
            and self.status != self.Status.COMPLETED

```

employees/models.py

```

from django.db import models
from django.conf import settings

class Employee(models.Model):
    user = models.OneToOneField(settings.AUTH_USER_MODEL,
                                on_delete=models.CASCADE,
                                null=True, blank=True)

    name = models.CharField(max_length=200)
    email = models.EmailField(unique=True)
    phone = models.CharField(max_length=20, blank=True)
    role_title = models.CharField(max_length=100,
                                  default='Architect')

    skills = models.TextField(blank=True,
                              help_text='Comma-separated skills')
    date_joined = models.DateField(auto_now_add=True)
    is_active = models.BooleanField(default=True)

    def __str__(self):
        return self.name

    def skills_list(self):
        return [s.strip() for s in self.skills.split(',') if s.strip()]

class ProjectAssignment(models.Model):
    project = models.ForeignKey('projects.Project',
                                on_delete=models.CASCADE,
                                related_name='assignments')

    employee = models.ForeignKey(Employee,
                                  on_delete=models.CASCADE,
                                  related_name='assignments')

    assigned_date = models.DateField(auto_now_add=True)
    notes = models.TextField(blank=True)

    class Meta:
        unique_together = ('project', 'employee')

    def __str__(self):
        return f"{self.employee.name} → {self.project.name}"

```

billing/models.py

```

from django.db import models
from django.conf import settings
from django.utils import timezone

class Invoice(models.Model):
    class Status(models.TextChoices):
        PENDING = 'PENDING', 'Pending'
        PAID = 'PAID', 'Paid'

```


6. Module Specifications

6.1 Authentication Module (app: accounts)

App name	accounts
URLs prefix	/accounts/
Key views	login_view, logout_view, profile_view
Templates	accounts/login.html, accounts/profile.html
Key logic	Role check decorators: @admin_required, @accountant_required

Decorator pattern to use across all views:

```
# accounts/decorators.py
from django.core.exceptions import PermissionDenied

def admin_required(view_func):
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated or not request.user.is_admin():
            raise PermissionDenied
        return view_func(request, *args, **kwargs)
    return wrapper
```

6.2 Project Management Module (app: projects)

App name	projects
URLs prefix	/projects/
Views	project_list, project_detail, project_create, project_update, project_delete, assign_architect
Templates	projects/list.html, projects/detail.html, projects/form.html
Permissions	Admin: full CRUD. Architect: view own + update phase. Accountant: read-only.

```
# projects/urls.py
urlpatterns = [
    path('', views.project_list, name='project-list'),
    path('create/', views.project_create, name='project-create'),
    path('<int:pk>/', views.project_detail, name='project-detail'),
    path('<int:pk>/edit/', views.project_update, name='project-update'),
    path('<int:pk>/delete/', views.project_delete, name='project-delete'),
    path('<int:pk>/assign/', views.assign_architect, name='project-assign'),
]
```

6.3 Employee Management Module (app: employees)

App name	employees
URLs prefix	/employees/
Views	employee_list, employee_detail, employee_create, employee_update, employee_delete
Templates	employees/list.html, employees/detail.html, employees/form.html
Permissions	Admin only for create/edit/delete. All roles can view list.

6.4 Billing Module (app: billing)

App name	billing
URLs prefix	/billing/
Views	invoice_list, invoice_detail, invoice_create, invoice_update, mark_paid
Templates	billing/list.html, billing/detail.html, billing/form.html
Permissions	Admin + Accountant: full access. Architect: view only.
Auto-logic	Invoice.save() auto-sets OVERDUE if due_date < today and status == PENDING

6.5 Dashboard Module (app: core)

App name	core
URL	/ (root)
Stats cards	Total Projects, Active Projects, Total Architects, Pending Invoices, Overdue Invoices
Charts	Monthly revenue bar chart (Chart.js), Project status doughnut chart
Tables	5 most recent projects, 5 upcoming deadlines, 5 recent invoices

7. Complete URL & View Reference

URL Pattern	View Function	Method	Permission	Description
/	core.views.dashboard	GET	Logged in	Main dashboard
/accounts/login/	accounts.views.login_view	GET/POST	Public	Login page
/accounts/logout/	accounts.views.logout_view	POST	Logged in	Logout
/projects/	projects.views.project_list	GET	All roles	List all projects
/projects/create/	projects.views.project_create	GET/POST	Admin	Add project
/projects/<pk>/	projects.views.project_detail	GET	All roles	Project detail
/projects/<pk>/edit/	projects.views.project_update	GET/POST	Admin	Edit project
/projects/<pk>/delete/	projects.views.project_delete	POST	Admin	Delete project
/projects/<pk>/assign/	projects.views.assign_architect	GET/POST	Admin	Assign architect
/employees/	employees.views.employee_list	GET	All roles	List architects
/employees/create/	employees.views.employee_create	GET/POST	Admin	Add architect
/employees/<pk>/	employees.views.employee_detail	GET	All roles	Architect detail
/employees/<pk>/edit/	employees.views.employee_update	GET/POST	Admin	Edit architect
/billing/	billing.views.invoice_list	GET	Admin/Acct	List invoices
/billing/create/	billing.views.invoice_create	GET/POST	Admin/Acct	Create invoice
/billing/<pk>/	billing.views.invoice_detail	GET	Admin/Acct	Invoice detail
/billing/<pk>/paid/	billing.views.mark_paid	POST	Admin/Acct	Mark as paid

8. Project File Structure

```
archportal/                                ← Django project root
├──
│   ├── manage.py
│   ├── requirements.txt
│   ├── .env                               ← SECRET_KEY, DB credentials (never commit)
│   ├── .env.example                       ← Template for .env
│   └──
│       ├── archportal/                   ← Project config package
│       │   ├── __init__.py
│       │   ├── settings.py               ← All settings, reads from .env
│       │   ├── urls.py                   ← Root URL config
│       │   ├── wsgi.py
│       │   └── asgi.py
│       └──
│           ├── accounts/                 ← Auth app
│           │   ├── migrations/
│           │   ├── templates/accounts/
│           │   │   ├── login.html
│           │   │   └── profile.html
│           │   ├── __init__.py
│           │   ├── admin.py
│           │   ├── apps.py
│           │   ├── decorators.py         ← Role-check decorators
│           │   ├── forms.py              ← LoginForm, UserProfileForm
│           │   ├── models.py             ← Custom User model
│           │   ├── urls.py
│           │   └── views.py
│           └──
│               ├── core/                 ← Dashboard app
│               │   ├── migrations/
│               │   ├── templates/core/
│               │   │   ├── dashboard.html
│               │   ├── templatetags/
│               │   │   ├── core_extras.py ← Custom template filters
│               │   ├── context_processors.py ← Global context (unread invoices etc)
│               │   ├── urls.py
│               │   └── views.py
│               └──
│                   ├── projects/         ← Projects app
│                   │   ├── migrations/
│                   │   ├── templates/projects/
│                   │   │   ├── list.html
│                   │   │   ├── detail.html
│                   │   │   └── form.html
│                   │   ├── forms.py
│                   │   ├── models.py
│                   │   ├── urls.py
│                   │   └── views.py
│                   └──
│                       ├── employees/    ← HR / Architect app
│                       │   ├── migrations/
│                       │   ├── templates/employees/
│                       │   │   ├── list.html
│                       │   │   ├── detail.html
│                       │   │   └── form.html
```

```

■   ■■■ forms.py
■   ■■■ models.py
■   ■■■ urls.py
■   ■■■ views.py
■
■■■ billing/                                ← Invoicing app
■   ■■■ migrations/
■   ■■■ templates/billing/
■   ■   ■■■ list.html
■   ■   ■■■ detail.html
■   ■   ■■■ form.html
■   ■■■ forms.py
■   ■■■ models.py
■   ■■■ urls.py
■   ■■■ views.py
■
■■■ templates/                              ← Shared / base templates
    ■■■ base.html                          ← Main layout with sidebar
    ■■■ partials/
    ■   ■■■ _sidebar.html
    ■   ■■■ _navbar.html
    ■   ■■■ _messages.html                ← Django messages flash alerts
    ■   ■■■ _pagination.html
    ■■■ errors/
        ■■■ 403.html
        ■■■ 404.html
        ■■■ 500.html

```

requirements.txt

```

Django>=5.0,<6.0
psycpg2-binary>=2.9
python-decouple>=3.8
whitenoise>=6.6
django-crispy-forms>=2.1
crispy-tailwind>=1.0
Pillow>=10.0

```

settings.py key configuration

```

# archportal/settings.py
from decouple import config

AUTH_USER_MODEL = 'accounts.User'  # MUST be set before first migration

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    # ... defaults ...
    'crispy_forms',
    'crispy_tailwind',
    'accounts',
    'core',
    'projects',
    'employees',
    'billing',
]

CRISPY_ALLOWED_TEMPLATE_PACKS = "tailwind"

```

9. Implementation Roadmap

Phase 1 — Project Bootstrap (Day 1)

- Create virtual environment: `python -m venv venv && source venv/bin/activate`
- Install requirements: `pip install -r requirements.txt`
- Create Django project: `django-admin startproject archportal .`
- Create all 4 apps: `python manage.py startapp accounts` (repeat for core, projects, employees, billing)
- Configure settings.py — AUTH_USER_MODEL, DATABASES, INSTALLED_APPS
- Create .env file with DB credentials and SECRET_KEY
- Run initial migration: `python manage.py makemigrations && python manage.py migrate`
- Create superuser: `python manage.py createsuperuser`

Phase 2 — Auth & Base Template (Day 1–2)

- Build custom User model in accounts/models.py
- Create base.html with sidebar navigation, responsive layout
- Build login/logout views and templates
- Implement role-check decorators in accounts/decorators.py
- Test role-based redirects

Phase 3 — Projects Module (Day 2–3)

- Define Project model and run migrations
- Build ProjectForm with all fields and validation
- Create list view with status filter and search
- Create detail view showing assigned architects and invoices
- Build create/update/delete views with admin-only access
- Add assign architect modal on project detail page

Phase 4 — Employees Module (Day 3–4)

- Define Employee and ProjectAssignment models
- Build employee CRUD views
- Add skills display as badge tags
- Show assigned projects on employee detail page

Phase 5 — Billing Module (Day 4–5)

- Define Invoice model with auto-overdue logic in save()
- Build invoice list with status colour coding
- Create invoice form linked to projects
- Add one-click 'Mark as Paid' button
- Add overdue invoice count to sidebar badge

Phase 6 — Dashboard & Polish (Day 5–6)

- Build dashboard view with aggregate queries
- Integrate Chart.js for revenue bar chart
- Add doughnut chart for project status breakdown
- Add recent projects table and upcoming deadlines table
- Test all role-based permissions end-to-end
- Add Django messages for all form submissions
- Handle 403 / 404 / 500 error pages

10. Claude Code — Step-by-Step Implementation Instructions

This section is written specifically for Claude Code. Follow these instructions in order. Each instruction block is a self-contained task.

STEP 1 — Environment Setup

```
mkdir archportal && cd archportal
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install Django psycopg2-binary python-decouple whitenoise django-crispy-forms crispy-tailwind
pip freeze > requirements.txt
django-admin startproject archportal .
python manage.py startapp accounts
python manage.py startapp core
python manage.py startapp projects
python manage.py startapp employees
python manage.py startapp billing
```

STEP 2 — .env File

```
SECRET_KEY=your-secret-key-here-generate-with-python-secrets
DEBUG=True
DB_NAME=archportal_db
DB_USER=postgres
DB_PASSWORD=yourpassword
DB_HOST=localhost
DB_PORT=5432
```

STEP 3 — PostgreSQL Setup

```
psql -U postgres
CREATE DATABASE archportal_db;
CREATE USER archportal_user WITH PASSWORD 'yourpassword';
GRANT ALL PRIVILEGES ON DATABASE archportal_db TO archportal_user;
\q
```

STEP 4 — After All Models Written

```
python manage.py makemigrations accounts
python manage.py makemigrations projects
python manage.py makemigrations employees
python manage.py makemigrations billing
python manage.py migrate
python manage.py createsuperuser
```

STEP 5 — Run Dev Server

```
python manage.py runserver
# Visit: http://127.0.0.1:8000/
```

STEP 6 — Create Test Users via Django Shell

```
python manage.py shell
>>> from accounts.models import User
```

```
>>> User.objects.create_user("architect1", "arch@test.com", "pass123", role="ARCHITECT")
>>> User.objects.create_user("accountant1", "acct@test.com", "pass123", role="ACCOUNTANT")
>>> exit()
```

Key Implementation Notes for Claude Code

- **AUTH_USER_MODEL** must be set **BEFORE** the first migration. If you run migrate before setting this, delete the db and all migration files (except `__init__.py`) and start over.
- **Template inheritance:** Every template extends `base.html`. The base has `{% block content %}` and `{% block title %}`.
- **CSRF tokens:** Every POST form must have `{% csrf_token %}` inside the `<form>` tag.
- **Messages framework:** Add `messages.success(request, 'Project created.')` in every successful view, and render with `{% if messages %}` in `base.html`.
- **login_required decorator:** Apply `@login_required` to every view except the login page.
- **Overdue invoices:** Run a management command or use a signal to auto-update overdue status daily. Simplest approach: call `Invoice.save()` check on every `invoice_list` view load.
- **Chart.js data:** Pass chart data from view to template as JSON: `json_script` template tag or a small `<script>` block with `{{ data|safe }}`.
- **Static files:** Add `STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'` and `'whitenoise.middleware.WhiteNoiseMiddleware'` to `MIDDLEWARE`.